Extracting Data from PDFs

Week 9 · Documents module · Textbook Chapter 9

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

October 12, 14 & 16, 2026

This week at a glance

PDFs are the web's filing cabinets. This week we pry official records out of them and turn locked documents into data.

Monday | Concepts & notebook intro

- Why PDFs resist extraction; reconstruction vs. reading

Wednesday | Notebook lab

- Work the Chapter 9 notebook: PyPDF, regex cleanup, Boulder City Council minutes

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 9

Companion notebook

`ch-09-pdfs.ipynb`

Bring a laptop

Wednesday is hands-on — we extract and debug live.

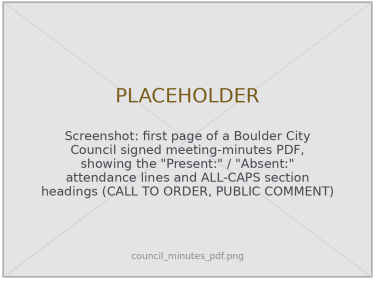
1. Monday · Concepts

The web's filing cabinets

PDFs are everywhere in institutional record-keeping: council minutes, bill texts, financial reports, court filings, academic papers, environmental impact assessments.

They are the format of choice for "official" documents because they **preserve visual layout** across any device or printer. But that priority — visual fidelity over semantic structure — makes them **hostile to programmatic extraction**.

The records locked inside are *readymade* data (Salganik (2018)): created for administration, not research, yet repurposable — with effort — to answer questions their creators never anticipated.



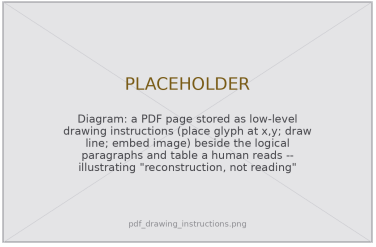
Boulder City Council minutes — our target this week.

A computer never reads a PDF

When **you** read a PDF you see paragraphs, tables, headers, and page numbers.

When a **computer** reads one it sees a sequence of drawing instructions: *place this character at* (x, y)*; draw a line from here to there; embed this image.*

So text extraction is **reconstruction, not reading —** the computer must infer logical structure from spatial layout. The results will not be perfect, but they can be good enough to turn inaccessible records into analyzable datasets.



Glyphs at coordinates, not sentences — structure is inferred.

Textbook Ch. 9, "The Web's Filing Cabinets."

Soft enclosure & the infrastructure of transparency

When governments publish minutes, budgets, and filings *only* as PDFs, they create what the book's post-API chapter calls **soft enclosure**: records that are nominally public but practically inaccessible for analysis.

Building extraction skills enacts the **oversight** and **ownership** values — public records belong to the public, and technical barriers are themselves a form of enclosure.

Adobe created the PDF in 1993 for visual fidelity across devices — at the cost of semantic accessibility. **DocumentCloud** (ProPublica & IRE), **MuckRock** (automating FOIA), and the **Sunlight Foundation** built the infrastructure that bridges public documents and accessible data.

DocumentCloud: shared infrastructure for analyzing document dumps.

Textbook Ch. 9, "Public Interest Connection" & "Social History and Public Interest."

What the notebook covers

This week's companion notebook builds one skill at a time:

- **PyPDF basics —** open a file, read metadata, extract page-level text (and see how messy it is)
- **Cleanup —** normalize whitespace *without* destroying line structure; use regex to impose fields
- **Case study —** Boulder City Council minutes: council attendance and public-comment counts
- **Pipeline —** a reusable class that processes a whole corpus of PDFs into a DataFrame

Connecting to Ch. 7 & the post-API theme

Once text is out, the **bag-of-words** preprocessing from the archives chapter (Ch. 7) sharpens topic tracking across many meetings. And PDF-only publishing *is* the "soft enclosure" the post-API chapter warns about — extraction is how we push back.

2. Wednesday · Notebook Lab

PyPDF basics — open, inspect, extract

`pypdf` gives you document metadata and page-level text. Start by looking at what actually comes out:

```
from pypdf import PdfReader

reader = PdfReader("council_minutes_2024_01.pdf")
print(len(reader.pages), "pages")
print(reader.metadata) # title, author, creation date...

page = reader.pages[0]
text = page.extract_text()
print(text[:500]) # expect mess: headers/footers mixed in, tables garbled,
                  # line breaks that don't match paragraphs
```

Textbook Ch. 9, "PyPDF Basics." Messy output is normal — cleaning it up is the core of the workflow.

The one line that flattens the page

Tempting one-liner:

```
# DON'T: \s matches newlines too,
# so this flattens the whole page
re.sub(r'\s+', ' ', raw_text)
```

Correct — collapse spaces & tabs, **keep** the line breaks:

```
text = re.sub(r'[ \t]+', ' ', raw_text)
lines = [ln.strip() for ln in text.split('\n')]
clean = '\n'.join(ln for ln in lines if ln)
```

The most important choice is what *not* to collapse.

Line structure is real information: section headings are recognizable **precisely because they sit alone on their own lines**.

Erase the newlines and no heading detector can ever find them.

Textbook Ch. 9, "Cleanup Strategies."

A regex toolkit for PDF text

Structured data hides inside a wall of characters; regex carves it out. Five patterns cover most government PDFs:

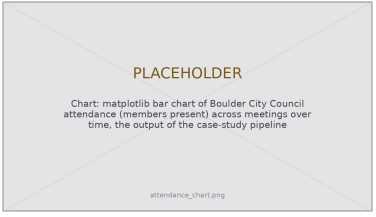
```
dates = re.findall(r'\d{1,2}/\d{1,2}/\d{2,4}', text) # 1/15/2024, 12/03/23
amounts = re.findall(r'\$[\d,]+\.?\d*', text) # $1,250.00, $3,400,000
emails = re.findall(r'[\w.+-]+@[\w-]+\.[\w.-]+', text) # clerk@bouldercolorado.gov

# ALL-CAPS section headings on their own line (a literal space, NOT \s,
# so one match can't swallow several consecutive heading lines):
headings = re.findall(r'^[A-Z][A-Z0-9 .:&-]{5,}$', text, re.MULTILINE)
# -> ['CALL TO ORDER', 'PUBLIC COMMENT', 'CONSENT AGENDA', 'ADJOURNMENT']
```

Each pattern encodes an assumption (slash dates, a leading $, uppercase headings). **Test across multiple pages and documents** before trusting a pipeline. New to regex? See *Missing Manual* Ch. 18.

Textbook Ch. 9, "Regular Expressions for PDF Text."

Case study — council attendance

```
def extract_attendance(pdf_path):
    text = PdfReader(pdf_path).pages[0].extract_text()
    # names follow "Present:" up to "Absent:" or end of line
    present = re.search(r'Present:\s*(.*?)\s*(?:Absent:|$)',
                        text, re.IGNORECASE | re.MULTILINE)
    absent = re.search(r'Absent:\s*(.*)$',
                       text, re.IGNORECASE | re.MULTILINE)
    return {"present": split_names(present),
            "absent": split_names(absent)}
```



Counts across meetings become a trend — scale reveals what one PDF can't.

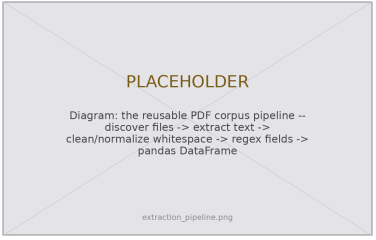
Be honest about limits: "Absent: None" becomes a phantom member named `None`; titles and separators vary. Treat output as a **first draft** — spot-check and patch.

Textbook Ch. 9, "Case Study: Boulder City Council Minutes." Filenames encode the date — parse them, not the PDF text.

Scale it — a reusable corpus pipeline

Wrap the workflow so one class processes a whole directory into a DataFrame:

```
class PDFCorpusAnalyzer:
    def __init__(self, directory, cleanup_patterns=None):
        self.files = sorted(f for f in os.listdir(directory)
                            if f.lower().endswith(".pdf"))
        self.cleanup_patterns = cleanup_patterns or []

    def process_all(self, extract_fn):
        rows = []
        for f in self.files:
            try: # one bad PDF won't kill the run
                text = self.extract_text(f)
                rows.append({**extract_fn(f, text), "file": f})
            except Exception as e:
                rows.append({"file": f, "error": str(e)})
        return pd.DataFrame(rows)
```



Discover → extract → clean → regex fields → DataFrame.

Textbook Ch. 9, "Building a Reusable PDF Pipeline." Pass document-specific cleanup patterns in; wrap each file defensively.

When PyPDF is not enough

PyPDF handles text-based PDFs well, but two common cases defeat it.

Tables — PyPDF flattens them into a text stream, losing rows and columns. `pdfplumber` preserves the grid:

```
import pdfplumber
with pdfplumber.open("report.pdf") as pdf:
    table = pdf.pages[0].extract_table() # a list of row-lists (cells)
    df = pd.DataFrame(table[1:], columns=table[0])
```

Scanned PDFs — a scan has no text layer, so PyPDF returns empty or garbled output. You need **OCR**:

```
# pytesseract + the Tesseract engine turn page images into text;
# accuracy varies with scan quality. (camelot is another table option.)
```

Know your tool's limits: reach for `pdfplumber`, `camelot`, or OCR when the layout demands it.

Textbook Ch. 9, "When PyPDF Is Not Enough" & "Common Issues to Debug."

Lab: work these, then show-and-tell

Work through the notebook exercises in pairs:

1. Find & extract one consistent element across $\geq$ `10` government PDFs $\rightarrow$ a DataFrame
2. Compare PyPDF quality on a text report, a table-heavy doc, and a slide deck
3. Tables, not text: extract a budget table with `pdfplumber` vs. `pypdf`; build numeric columns
4. Topic tracking: count a term ("housing," "police," "flood") across a year of minutes; plot spikes (reuse Ch. 7 bag-of-words)
5. Tool comparison: `pdfplumber` vs. `pypdf` on the same PDF
6. 5617: quantify extraction quality across two cities' PDFs; write a data-quality memo using Gebru et al. (2021) categories

Show-and-Tell

Come ready to share:

- a challenging bug
- interesting data you pulled
- a provocation for a research design

PDFs are built for eyes, not for code.
Every extraction is an educated reconstruction —
never perfect, often good enough.

Imprecision is the price of scale —
and scale is what turns documents into data.

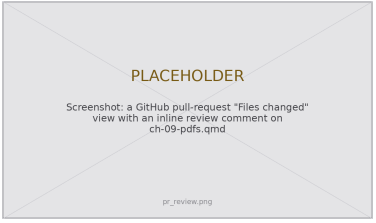
3. Friday · Textbook Revisions

Improving Chapter 9 together

Today we make the book better. Each of you opens a **pull request** against `ch-09-pdfs.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable



Contributions & reviews count toward the Textbook Revisions grade (30%).

A revision menu for Chapter 9

High-value targets — pick one and scope it to a single PR:

- **Run it against real Boulder minutes.** The `# Returns:` outputs and five regex patterns are *illustrative*. Download actual signed minutes, report what the patterns catch and miss, and label sample output as representative.
- **Add the chapter's first figures.** It is all code and prose today: a "drawing-instructions vs. logical text" diagram, a pipeline flowchart, or an annotated minutes screenshot would anchor Monday's core idea.
- **Harden `extract_attendance`.** The chapter itself flags "Absent: None," title variants ("Council Member Smith"), and semicolon / line-break separators. Contribute handling **plus tests** for the formats you actually hit.
- **Make an exercise self-checking.** Add expected output, a rubric, or a starter cell to the open-ended "find and extract" and "topic tracking" exercises.
- **Deepen "When PyPDF Is Not Enough."** Turn the two-line `pdfplumber` / OCR sketches into a runnable table-extraction or Tesseract example tied to "Further Reading."

Targets drawn from the chapter's callouts, "Common Issues," exercises, & "Further Reading."

What makes a good revision & a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: a new module on **APIs** — when data arrives as JSON and no parsing is required (starting with Wikipedia).

Key takeaways

1. PDFs optimize **visual fidelity over structure —** extraction is reconstruction, not reading.
2. `pypdf` gives page-level text & metadata; expect messy output and plan to clean it.
3. Collapse spaces/tabs but **keep newlines —** line structure is information; regex imposes the rest.
4. Abstract into a **pipeline** so one bad file doesn't kill a corpus run; test patterns across documents.
5. Know PyPDF's limits: `pdfplumber` for tables, OCR for scans. Imprecision is the price of scale.

Reading: Textbook Ch. 9. Related: Salganik (2018); Gebru et al. (2021).